INSTALL & HANDOVER GUIDE

QuixiDesk on Android

A pocket assistant that runs the engine on the phone itself — no computer required, no account required for what it does offline.

Written for whoever is holding the phone: every step is a screen you can reach, with the exact thing to touch boxed in yellow. The last part is written for the assistant itself, so it can find its own files, its logs, and its dependencies.

Version — built from the current tree
Phone shown — Android 16 (emulator). Samsung and Pixel wording differs slightly and is noted where it does.
Companion — the Windows 11 laptop, used only for sync. Nothing here needs it.

What is in this guide

1 · What you are installing 3

2 · Before you start — what the phone needs 3

3 · Getting the app file onto the phone 4

4 · Allowing the install 4

5 · Installing and opening it 5

6 · Turning on Developer options 6

7 · USB debugging 8

8 · Wireless debugging 9

9 · Stop Android from closing it 10

10 · Give it a brain: your model service 10

11 · Signing in to mail, once 10

12 · Android Auto 12

13 · When something goes wrong 12

A · Appendix for the assistant: files, dependencies, logs 13

C · The Windows side 17

B · Undoing all of it 17

1 · What you are installing

QuixiDesk is one app file. Everything it needs to think, remember, and use tools is inside that file — the engine, the persona, the skills. It does not call home, and it does not need a laptop running in the next room.

Two things it may reach for, and only when you tell it to:

- **A model service** — the “brain”. Either a key for a hosted service, or a machine of your own on the same Wi-Fi. Without one it opens and shows its screens but cannot hold a conversation.
- **Your mail** — only if you sign in. Email is read by driving the mail website inside the app, the way a person would, because free Proton accounts do not allow the app to connect directly.

2 · Before you start

What	Why, and how much
Android 10 or newer	The app draws itself with the phone’s built-in web view. Android 10 is the oldest version with the pieces it needs.
About 1 GB free	The app file is about 110 MB; plan for roughly twice that once it is installed. The rest is room for its replies and any files it writes.
A model service	A key from one of the services listed inside the app, *or* the address of your own engine on the same network.
Optional: a Proton account	Only if you want mail in the app. The free tier is fine.

> **You do not need:** a Google account, a computer, or a cable — unless you want to use the debugging features in parts 7, 8 and A.

3 · Getting the app file onto the phone

The file ends in `.apk`. Any of these work; pick whichever you already do.

- **By email or chat** — send the file to yourself and tap it in the message. Fastest for one phone.
- **By USB cable** — plug in, choose *File transfer* on the phone, and drop the file into the phone's *Download* folder.
- **By the browser** — download it directly on the phone, if it is hosted somewhere you can reach.

Wherever it lands, it will be in **Files → Downloads**.

If the computer is a Windows PC

1. Plug the phone into the PC with the charging cable.
2. On the phone, pull down the notification shade and tap the USB notification. Choose **File transfer** — *not* "Charging only", which is the default and hides the phone from the PC.
3. In File Explorer the phone appears under **This PC**. Open it → **Internal storage** → **Download**, and copy the `.apk` and this guide there.
4. Eject safely, unplug, then on the phone open **Files → Downloads** and tap the file.

> **Nothing needs installing on Windows.** No driver, no companion app — the phone appears as a normal drive. If it does not appear, the usual cause is a charge-only cable, or the USB mode still set to "Charging only".

Which file is for your phone

File	Use it for
`QuixiDesk-public-arm64.apk`	Every normal Android phone made since roughly 2017 — Pixel, Samsung, OnePlus, Motorola, Xiaomi, and so on.
a build ending `x86_64`	Emulators and a few Chromebooks. It will refuse to install on a phone with *INSTALL_FAILED_NO_MATCHING_ABIS* — that message means you have the wrong file, not a broken one.

4 · Allowing the install

Android blocks apps that did not come from the Play Store until you say otherwise **for the app that is holding the file** — Chrome, Files, Gmail, whichever it is.

1. Tap the `.apk` file.
2. If a warning appears — *"For your security, your phone is not allowed to install unknown apps from this source"* — tap **Settings** on that warning.
3. Turn on **Allow from this source**, then press back.
4. Tap the file again and choose **Install**.

> **Samsung phones: there is a second gate.** Auto Blocker (Settings → Security and privacy → Auto Blocker) refuses apps from outside the store at the system level, and it also refuses `adb install`. If your file will not install and nothing on the warning mentions Auto Blocker, that is what is stopping it. Turn Auto Blocker off for the install, and turn it back on afterwards if you want it — the app does not need it off once installed.

> **If the "Install unknown apps" list says "No apps", that is normal.** The list only fills in with apps that have actually tried to install something. Tap your file first, and the app that holds it appears there.

5 · Installing and opening it

Play Protect may offer to scan the file. It is fine to let it, or to continue without scanning. When it finishes, tap **Open**.

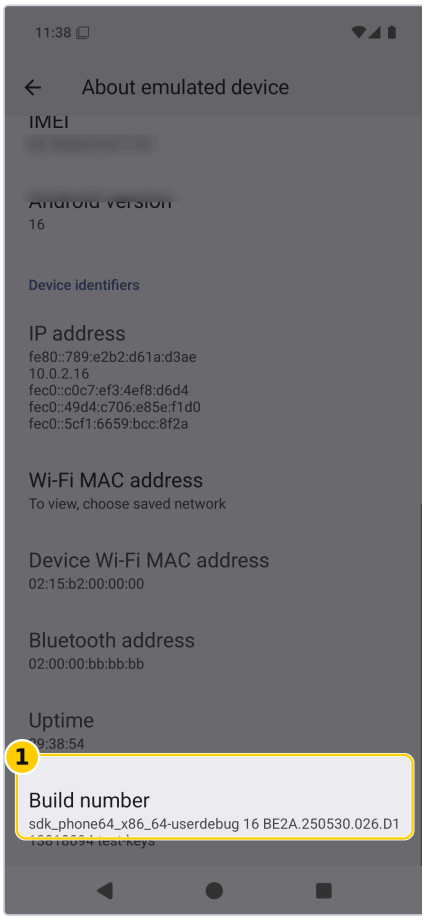
First launch takes a few seconds longer than later ones: the app writes its persona and its skills into its own storage. You will see a greeting screen with a box at the bottom that says *“You gonna talk to me or what?”* — that is the app, ready.

6 · Turning on Developer options

“Developer options” is a hidden settings menu that Android keeps out of the way. Everything in parts 7 and 8 lives there. Turning it on changes nothing by itself — it only makes the menu visible.

1 Open Settings → About phone

On Samsung it is **Settings → About phone**; on Pixel it is **Settings → About phone → Device details**. Scroll to the very bottom, past the network details, until you see **Build number**.



1 — Build number. The last row on the About screen. Tap this seven times. On the phone used here the screen is called “About emulated device”; on a real phone it says “About phone”.

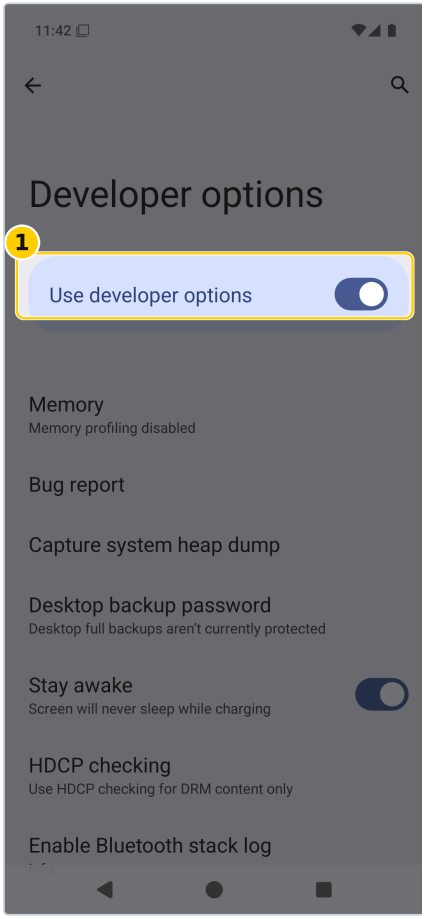
2 Tap “Build number” seven times

Tap it seven times in a row. After the third tap Android starts counting down: *“You are now 4 steps away from being a developer.”* Keep going. When you reach zero it says **“You are now a developer.”**

> If your phone asks for your PIN or pattern at this point, that is the phone confirming it is you. It is not a sign that something went wrong.

3 Find the new menu

Settings → System → Developer options (Samsung: **Settings → Developer options**). Make sure the switch at the top, **Use developer options**, is on.



2 — The master switch. Everything below this row only exists while it is on. Turning it off later hides the whole menu again without undoing anything else.

7 · USB debugging

This is what lets a computer (or a person helping you) look inside the phone: read the app’s log, copy its files out, install a newer build over the top without losing anything. On its own it does nothing.

4

The fast way to find it

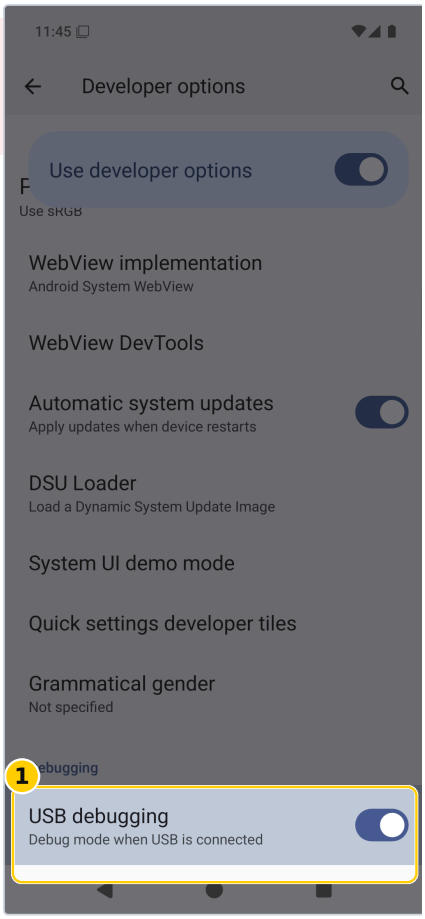
Developer options is a long list. Rather than scrolling, use the search at the top of Settings:



5

Turn it on

> **Turn it off when you are done.** While it is on, anyone who plugs in a cable can read the phone. It is a workshop door, not a front door — open it when you are working, close it after.



3 — USB debugging. Under the "Debugging" heading, subtitle *"Debug mode when USB is connected"*. Slide it on; Android asks you to confirm once.

8 · Wireless debugging

Same menu, a few rows below USB debugging. This lets a computer on the same Wi-Fi talk to the phone with no cable at all — which is how the phone gets updated while it sits in a pocket.

1. Turn on **Wireless debugging**.
2. Tap **Pair device with pairing code**. It shows an address like `192.168.1.50:37105` and a six-digit code.
3. On the computer, in that same Wi-Fi network, run `adb pair 192.168.1.50:37105` and type the code when asked. From then on the computer connects with `adb connect 192.168.1.50:5555`.

> Both devices must be on the same network, and the phone must stay awake while pairing. If the code expires, close the dialog and tap the pairing option again for a fresh one.

9 · Stop Android from closing it

Android aggressively closes apps that are not on screen. An assistant that gets killed in the background forgets what it was doing, so give it an exemption.

1. **Settings → Apps → QuixiDesk**.
2. **Samsung:** Battery → **Unrestricted**. **Pixel:** App battery usage → **Unrestricted**.
3. Optional, Samsung only: **Settings → Battery → Background usage limits** and remove QuixiDesk from "Sleeping apps" if it appears there.

10 · Give it a brain: your model service

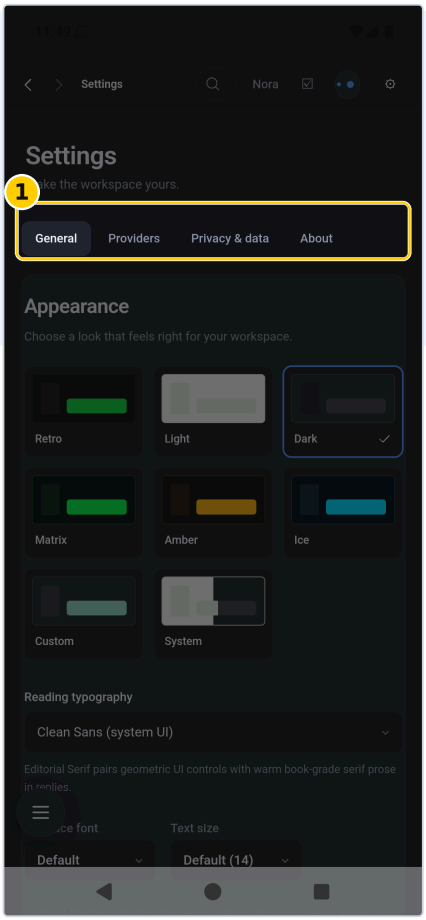
The app ships with its engine and its persona, but a model — the thing that composes sentences — is yours to point at. Open the gear at the top right of the app.

6

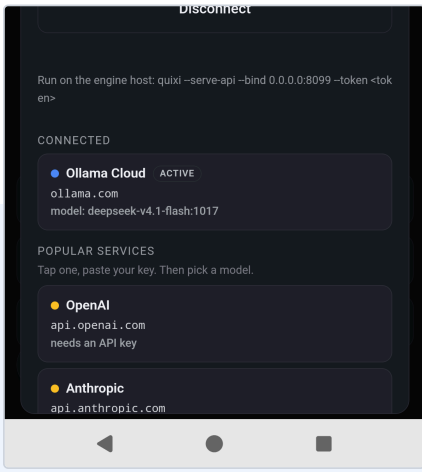
Pick a service, paste a key, choose a model

If you run your own engine on a computer at home, use the **Server** and **Engine token** fields at the top instead of a service key: the address looks like `http://192.168.1.50:8099`, and the token is the one that machine was started with.

> **Which model?**
> Anything that can hold a conversation works. Choose the largest one you are willing to pay for; the small ones answer quickly and forget the middle of a long conversation.



5 — The Providers tab. Inside the app: gear → Settings → Providers. This is the screen to return to whenever a key needs changing or a second model added.



4 — What is connected, and what could be. The top card names the service in use and the exact model. Under **Popular services**, tap a name, paste the key the service gave you, then pick a model from the list it returns.

11 · Signing in to mail, once

Email is optional. When you ask the assistant to read your mail the first time, the app opens the mail website inside itself and waits for you to sign in.

1. Type your address, tap **Continue**, then your password.
2. Complete whatever check the site shows — a picture puzzle, or the “I am human” box.
3. When your inbox appears, you are done. The app keeps that sign-in, so this happens once, not every time.

> **Never type your password into the app itself.** It is only ever typed into the mail site’s own page. If a screen that is not your mail provider asks for it, close it.

> Free Proton accounts cannot be read by apps the normal way — that is why the app drives the page instead. It means the mail screens look exactly like the website when they are being read, then hand you back to your conversation.

12 · Android Auto

If the phone is going to be used in a car, Android Auto has its own hidden developer menu. You need the **Android Auto** app installed and the phone connected to the car once before these options appear.

1. **Settings → Connected devices → Android Auto**.
2. Scroll to the bottom and tap **Version** ten times, until a message says developer mode is enabled.
3. Open the three-dot menu → **Developer settings**.
4. For a car that is not recognising the phone: enable **Add new cars to Android Auto** and, while connected by cable, **Unknown sources**.

> **Honesty about this part:** the screenshots in this guide were taken on a phone without Android Auto installed, so this section is written from the menu path rather than photographed. The wording above matches the current Android Auto app; if a label has moved, the three-dot menu at the top right of Android Auto's own settings is where it will be.

13 · When something goes wrong

What you see	What it means, and what to do
"App not installed"	An older copy is already there. Uninstall the old one first, or install the newer file over the top — your conversations and persona are kept.
"Install unknown apps" is empty	Normal. Tap the `.apk` first; the app holding it then appears in that list.
It opens but will not answer	No model service is connected. Gear → pick a service or enter your own server.
It answers, then forgets	Android is killing it in the background. Part 9.
Mail says it cannot read the inbox	You are signed out of the mail site. Ask it to check mail again; sign in on the page it opens.
The screen goes blank after tapping mail	The mail site is loading. If it stays blank for more than a minute, force-close the app and reopen; the conversation is still there.
Everything looks tiny	The app is designed for a phone screen; if text is small, raise **Settings → Display → Font size** in Android rather than zooming the app.

A · Appendix for the assistant: files, dependencies, logs

This part is written for the AI itself, or for whoever is helping it. The rest of the guide is for a person holding the phone.

Where everything lives

```
# the app package (release / debug builds differ only in the suffix)
org.quixidesk.android
org.quixidesk.android.debug

# its private state — persona, skills, workspace, mail bridge token
/data/user/0/org.quixidesk.android/state/persona.md
/data/user/0/org.quixidesk.android/state/skills/
/data/user/0/org.quixidesk.android/state/workspace/
/data/user/0/org.quixidesk.android/mail-bridge.token     # 0600, loopback only

# the UI it draws, inside the APK
assets/resources/ui/            # app.js, app.css, mobile.js, human.js
assets/resources/persona/nora-public.md
assets/resources/bundled-skills/
```

Building it

```
# toolchain (this machine)
JAVA_HOME=$HOME/jdk21
ANDROID_HOME=$HOME/Android/Sdk
NDK=$ANDROID_HOME/ndk/
RUSTUP_HOME=$HOME/win-cross/rustup   CARGO_HOME=$HOME/win-cross/cargo

# the app builds its own engine first: quixi-mobile is a static library inside the APK
cd /quixidesk-android
./build-apk.sh arm64 release          # or: ./build-apk.sh x86_64 debug  (emulator)
# artefact lands in dist/

adb install -r -d dist/QuixiDesk-arm64-release.apk
```

Cross-compiling the engine needs the NDK linker explicitly, or cargo silently uses the host compiler and the link fails:

```
CARGO_TARGET_AARCH64_LINUX_ANDROID_LINKER=$NDK/toolchains/llvm/prebuilt/linux-x86_64/bin/aarch64-linux-
android24-clang
```

Watching it work

```
# the three streams that matter
adb logcat -s app_lib:* Tauri/Console:* RustStdoutStderr:* AndroidRuntime:E
adb logcat -d | grep -E 'FATAL|Uncaught|mail driver install|persona '

# what the app thinks it is doing right now
adb shell dumpsys window | grep mCurrentFocus
adb shell dumpsys activity activities | grep mResumedActivity

# drive the UI from a machine (no touchscreen needed)
adb shell am start -n com.android.settings/.Settings\$DevelopmentSettingsActivity
adb shell uiautomator dump /sdcard/ui.xml    # NOTE: fails with "could not get idle state" on
                                             # some builds; fall back to screencap + eyeballs
adb exec-out screencap -p > shot.png

# the app's own webview, for inspecting the interface
```

```
adb forward tcp:9455 localabstract:webview_devtools_remote_$(adb shell pidof org.quixidesk.android.debug)
curl -s http://127.0.0.1:9455/json      # list targets before attaching
```

When it will not start

```
adb logcat -c && adb logcat | grep -E 'FATAL|dlopen|UnsatisfiedLink'
# state reset without uninstalling (loses persona edits, keeps the app)
adb shell pm clear org.quixidesk.android
# a build that must not be debuggable, installed beside the release one
adb install -r -d dist/QuixiDesk-arm64-debug.apk
```

Dependencies, stated plainly

- **On the phone:** nothing beyond Android itself. No Termux, no Python, no Node. The engine is compiled into the APK.
- **To build it:** Rust with the Android targets, the NDK, JDK 21, and the Tauri CLI.
- **Network:** only for the model service and mail. Nothing else leaves the phone unless you ask it to.

C · The Windows side

This is how the phone and a home computer keep the same memory. It works now, and it is deliberately boring: a small program on the PC, a folder on each machine, and one rule — **the newer copy wins, and the older one is never deleted**.

On the Windows computer, once

1. Install **Python 3** from python.org. That is the only requirement; there is nothing else to install.
2. Copy the folder `quixidesk-sync` somewhere permanent, for example `C:\QuixiDesk\sync`.
3. Open `windows\start-sync-server.cmd` in Notepad and change `TOKEN` to a long random string. Keep a copy of it.
4. Double-click `start-sync-server.cmd`. A window opens and stays open — that window *is* the sync service.
5. If Windows Firewall asks, allow it on **Private networks** only.

> The computer keeps its copy of the assistant's state in `%USERPROFILE%\QuixiDesk`. Memory, persona and preferences live there, and that is the folder to back up.

On the phone, once

The phone needs two values — the computer's address and that token — written into a small file in the app's own storage:

```
{
  "url":   "http://192.168.1.50:8788",
  "token": "the-same-long-string"
}
```

Whoever sets the phone up writes this once. After that, **the app syncs every time it opens** — no button, no schedule. A manifest of file hashes is a few hundred bytes, so the phone can check every launch and only move what actually changed.

What moves, and what never does

Syncs	Never syncs
Memory (what it knows about her)	Passwords, API keys, sign-ins
The persona	Conversations in progress
Preferences	Workspace files, artifacts, scratchpads

When both machines changed the same thing

Nothing is lost. The version that lost is saved next to the winner, named `<file>.conflict-<machine>-<time>`. It stays until someone deletes it, so a bad merge is always recoverable by hand.

What has actually been tested

Run against a real phone, in both directions: three memory files uploaded from the phone, a newer persona pulled down, an edit made on the computer then pulled on the next pass, and a second run immediately after moving nothing at all — which is what "converged" looks like:

```
pass 1   pushed 3 · pulled 1 · errors 0
pass 2   pushed 0 · pulled 0 · errors 0
```

What this is not: it is not a live connection. The phone syncs when it opens, not the instant it joins the Wi-Fi, because Android will not let an app watch the network without a permanent notification. If she edits on the phone and wants it on the computer immediately, close and reopen the app.

B · Undoing all of it

Every step here is reversible, and none of them need a computer.

To undo	Do this
Developer options	Settings → System → Developer options → turn **Use developer options** off. The menu hides; nothing else changes.
USB debugging	Same screen, slide **USB debugging** off. Turn this off whenever you are not using it.
Wireless debugging	Same screen, slide it off. Any computer that was paired loses access immediately.
Install-from-this-source permission	Settings → Apps → Special access → Install unknown apps → the app you allowed → turn it off.
Battery exemption	Settings → Apps → QuixiDesk → Battery → back to **Optimised**.
The app and everything in it	Long-press the icon → App info → Uninstall. Its persona, skills, conversations and saved sign-ins go with it.

> **Keep a copy of anything you want to keep first.** The app keeps conversations on the phone; uninstalling removes them.